# DATABASE MANAGEMENT SYSTEM LAB

# ASSIGNMENT NO.8

Name: Nishtha Pramod Parve

PRN:124B1F003

Case Study: Airline Management System

Mini Project: Centralized Club Management System

## Case Study: Airline Management System

**Aim:** No SQL: Aggregation and Indexing: Design and Develop Queries using aggregation and indexing with suitable example using Mongo DB.

Aggregation in **MongoDB** is used to **process data and return computed results** (like SUM, COUNT, AVG).

It works using **Aggregation Pipeline**, which includes stages like:

- $match → filter data
- $group → group data
- $sort → sort results
- $project → select fields

Indexing improves **query performance** by creating a fast lookup for data.

Without index → full collection scan (slow)
With index → fast search

**1. Aggregation Queries**

**(A) Count Total Bookings per Flight**

```
db.bookings.aggregate([
 {
  $group: {
   _id: "$flight_id",
   total_bookings: { $sum: 1 }
  }
 }
]);
```

```
airline> db.bookings.aggregate([
|    {
|      $group: {
|        _id: "$flight_id",
|        total_bookings: { $sum: 1 }
|      }
|    }
| ]);
```

**(B) Count Passengers by Travel Class**

```
db.bookings.aggregate([
 {
  $group: {
   _id: "$travel_class",
```

```
    total: { $sum: 1 }
  }
 }
]);
```

```
airline> db.bookings.aggregate([
|    {
|      $group: {
|        _id: "$travel_class",
|        total: { $sum: 1 }
|      }
|    }
| ]);
```

**(C) Filter + Group (Bookings after date)**

```
db.bookings.aggregate([
 {
  $match: {
   booking_date: { $gt: "2024-01-01" }
  }
 },
 {
  $group: {
   _id: "$flight_id",
   total: { $sum: 1 }
  }
 }
]);
```

```
airline> db.bookings.aggregate([
|    {
|      $match: {
|        booking_date: { $gt: "2024-01-01" }
|      }
|    },
|    {
|      $group: {
|        _id: "$flight_id",
|        total: { $sum: 1 }
|      }
|    }
| ]);
```

**(D) Join + Aggregation ($lookup)**

```
db.bookings.aggregate([
 {
```

```
    $lookup: {
      from: "passengers",
      localField: "passenger_id",
      foreignField: "passenger_id",
      as: "passenger_details"
    }
  },
  {
    $project: {
      booking_id: 1,
      "passenger_details.name": 1
    }
  }
]);
```

```
airline> db.bookings.aggregate([
|    {
|      $lookup: {
|        from: "passengers",
|        localField: "passenger_id",
|        foreignField: "passenger_id",
|        as: "passenger_details"
|      }
|    },
|    {
|      $project: {
|        booking_id: 1,
|        "passenger_details.name": 1
|      }
|    }
| ]);
```

**(E) Sort Flights by Number of Bookings**

```
db.bookings.aggregate([
  {
    $group: {
      _id: "$flight_id",
      total: { $sum: 1 }
    }
  },
  {
    $sort: { total: -1 }
  }
]);
```

```
airline> db.bookings.aggregate([
|    {
|      $group: {
|        _id: "$flight_id",
|        total: { $sum: 1 }
|      }
|    },
|    {
|      $sort: { total: -1 }
|    }
| ]);
```

**2. Indexing Queries**

**(A) Create Index on passenger_id**

db.bookings.createIndex({ passenger_id: 1 });

**(B) Create Index on flight_id**

db.bookings.createIndex({ flight_id: 1 });

**(C) Compound Index**

db.bookings.createIndex({ passenger_id: 1, flight_id: 1 });

**(D) View Indexes**

db.bookings.getIndexes();

**(E) Drop Index**

db.bookings.dropIndex({ passenger_id: 1 });

```
airline> db.bookings.createIndex({ passenger_id: 1 });
passenger_id_1
airline> db.bookings.createIndex({ flight_id: 1 });
flight_id_1
airline> db.bookings.createIndex({ passenger_id: 1, flight_id: 1 });
passenger_id_1_flight_id_1
airline> db.bookings.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { passenger_id: 1 }, name: 'passenger_id_1' },
  { v: 2, key: { flight_id: 1 }, name: 'flight_id_1' },
  {
    v: 2,
    key: { passenger_id: 1, flight_id: 1 },
    name: 'passenger_id_1_flight_id_1'
  }
]
airline> db.bookings.dropIndex({ passenger_id: 1 });
{ nIndexesWas: 4, ok: 1 }
```

# Mini Project: Centralized Club Management System

## 1. Aggregation Queries

### (A) Count Members in Each Club

```
db.club_members.aggregate([
  {
    $group: {
      _id: "$club_id",
      total_members: { $sum: 1 }
    }
  }
]);
```

```
club> db.club_members.aggregate([
|     {
|       $group: {
|         _id: "$club_id",
|         total_members: { $sum: 1 }
|       }
|     }
| ]);
```

### (B) Count Present Members

```
db.attendance.aggregate([
  {
    $match: { status: "Present" }
  },
  {
    $group: {
      _id: "$event_id",
      total_present: { $sum: 1 }
    }
  }
]);
```

```
club> db.attendance.aggregate([
|     {
|       $match: { status: "Present" }
|     },
|     {
|       $group: {
|         _id: "$event_id",
|         total_present: { $sum: 1 }
|       }
|     }
| ]);
[ { _id: 1, total_present: 1 } ]
```

**(C) Count Approved Budget Requests**

```
db.budget_request.aggregate([
  {
    $match: { approved_status: "Approved" }
  },
  {
    $group: {
      _id: "$club_id",
      total_approved: { $sum: 1 }
    }
  }
]);
```

```
club> db.budget_request.aggregate([
|     {
|       $match: { approved_status: "Approved" }
|     },
|     {
|       $group: {
|         _id: "$club_id",
|         total_approved: { $sum: 1 }
|       }
|     }
| ]);

club> |
```

**(D) Join Members with Club ($lookup)**

```
db.club_members.aggregate([
  {
    $lookup: {
      from: "club",
      localField: "club_id",
      foreignField: "club_id",
      as: "club_details"
```

```
    }
  },
  {
    $project: {
      m_name: 1,
      "club_details.club_name": 1
    }
  }
]);
```

```
club> db.club_members.aggregate([
|     {
|       $lookup: {
|         from: "club",
|         localField: "club_id",
|         foreignField: "club_id",
|         as: "club_details"
|       }
|     },
|     {
|       $project: {
|         m_name: 1,
|         "club_details.club_name": 1
|       }
|     }
| ]);
```

**(E) Sort Events by Date**

```
db.event.aggregate([
  {
    $sort: { start_date: 1 }
  }
]);
```

```
club> db.event.aggregate([
|   {
|     $sort: { start_date: 1 }
|   }
| ]);
[
  {
    _id: ObjectId('69dfc995139839ef423682d2'),
    event_id: 10,
    event_name: 'Tech Talk',
    club_id: 1,
    start_date: '2026-05-01'
  }
]
```

**2. Indexing Queries**

**(A) Create Index on club_id**

db.club_members.createIndex({ club_id: 1 });

**(B) Create Index on event_id**

db.attendance.createIndex({ event_id: 1 });

**(C) Compound Index**

db.attendance.createIndex({ member_id: 1, event_id: 1 });

**(D) View Indexes**

db.club_members.getIndexes();

**(E) Drop Index**

db.club_members.dropIndex({ club_id: 1 });

```
club> db.club_members.createIndex({ club_id: 1 });
club_id_1
club> db.attendance.createIndex({ event_id: 1 });
event_id_1
club> db.attendance.createIndex({ member_id: 1, event_id: 1 });
member_id_1_event_id_1
club> db.club_members.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { club_id: 1 }, name: 'club_id_1' }
]
club> db.club_members.dropIndex({ club_id: 1 });
{ nIndexesWas: 2, ok: 1 }
club>
```

## Conclusion

Aggregation in MongoDB plays a crucial role in processing and analyzing data efficiently in both the Airline Management System and the Centralized Club Management System. By using aggregation pipeline stages such as $match, $group, $sort, and $project, complex queries can be performed to generate useful insights like total bookings per flight, passenger

distribution, attendance tracking, and budget approvals. This enables better decision-making and data analysis in real-world scenarios.

Indexing further enhances the performance of these systems by providing faster access to frequently queried data, reducing the time required for searching and filtering records. It minimizes full collection scans and improves overall system efficiency, especially when dealing with large datasets.

Together, aggregation and indexing make MongoDB a powerful and scalable solution for both airline and club management systems. They ensure flexibility in handling dynamic data structures, improve query performance, and support efficient management of complex operations, making them highly suitable for modern database applications.